Implementing Conditional Access on FPGA

A dual-core, cache-coherent RISC-V processor with conditional access [1], on an AMD KR260

Aryan Kumar, Nishant Dhankhar, Kudumula Vinod Kumar Reddy *Advisor:* Ajay Singh

Department of Computer Science and Engineering, IIT Gandhinagar

Code and data

1 The problem: use-after-free

In a lock-free data structure, threads read shared nodes without taking locks while other threads remove them. If a removed node is freed while another core still holds a pointer to it, that core reads freed memory (Figure 1).

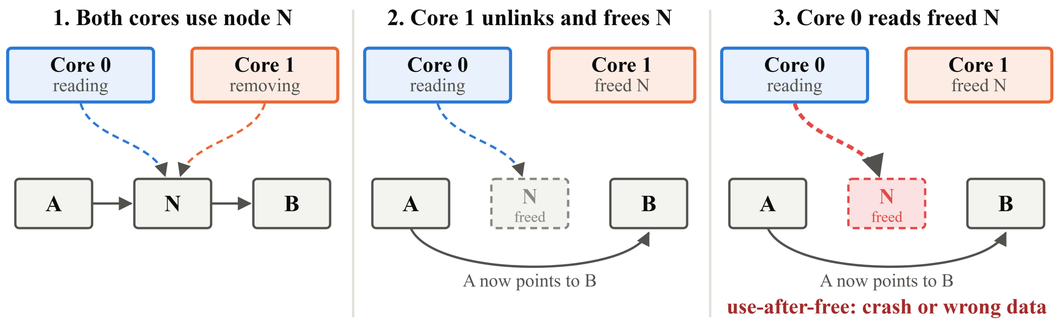


Figure 1. A use-after-free on a shared linked list.

2 Existing software approaches

- **Hazard pointers (HP)** [3]: a reader announces each pointer before using it, and a node is freed only when no announcement names it. *Problem:* every read pays a memory fence, which slows traversals.
- **Epoch-based reclamation (EBR)** [4]: removed nodes are freed in batches once every thread has passed a quiescent point. *Problem:* unfreed memory piles up, without bound if one thread stalls.

The common issue: delayed reclamation. Both schemes free a node only some time after it is removed, so they trade speed against memory.

3 Solution: Conditional Access

Conditional Access (CA) [1] adds memory instructions that *watch* the cache lines a thread reads. If another core writes one, the thread's next CA instruction *fails* and it retries. So a removed node can be freed **immediately**.

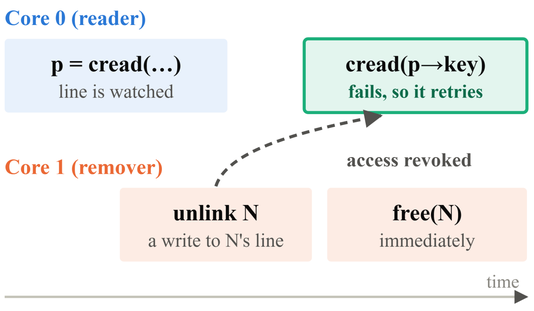


Figure 2. The remover's write revokes the reader's access, so the reader retries instead of using freed memory.

The four instructions of [1], which we built in hardware, and the one we added:

`cread`	load and watch the line
`cwrite`	store if nothing watched changed
`untagOne`	stop watching one line
`untagAll`	stop watching all lines
`bcafail`	jump if a CA op failed

What the software looks like

```
void pop(void) {                    // from our stack benchmark
retry: untagAll();
  node *t  = cread(&top);           // read top and watch its line
  if (bcafail()) goto retry;        // one branch: did a watched line change?
  node *nx = cread(&t->next);       // watch t as well
  if (bcafail()) goto retry;
  cwrite(&top, nx);                 // store only if top and t are unchanged
  if (bcafail()) goto retry;
  untagAll(); free(t);              // safe at once: any reader of t will fail
}
```

Figure 3. A stack pop with CA (simplified): no hazard pointers or epochs, the node is freed as soon as it is unlinked.

4 System design, on a real FPGA

CA was only ever evaluated in a simulator. Real hardware answers: **does it work**, **what does it cost**, and **can a simulator be trusted?** Starting from CVA6 [2], an open-source single-core RISC-V processor, we added **(1)** a second core, **(2)** MSI cache coherence through a directory, **(3)** the CA instructions plus `bcafail`, and **(4)** a shared L2 cache, as in [1].

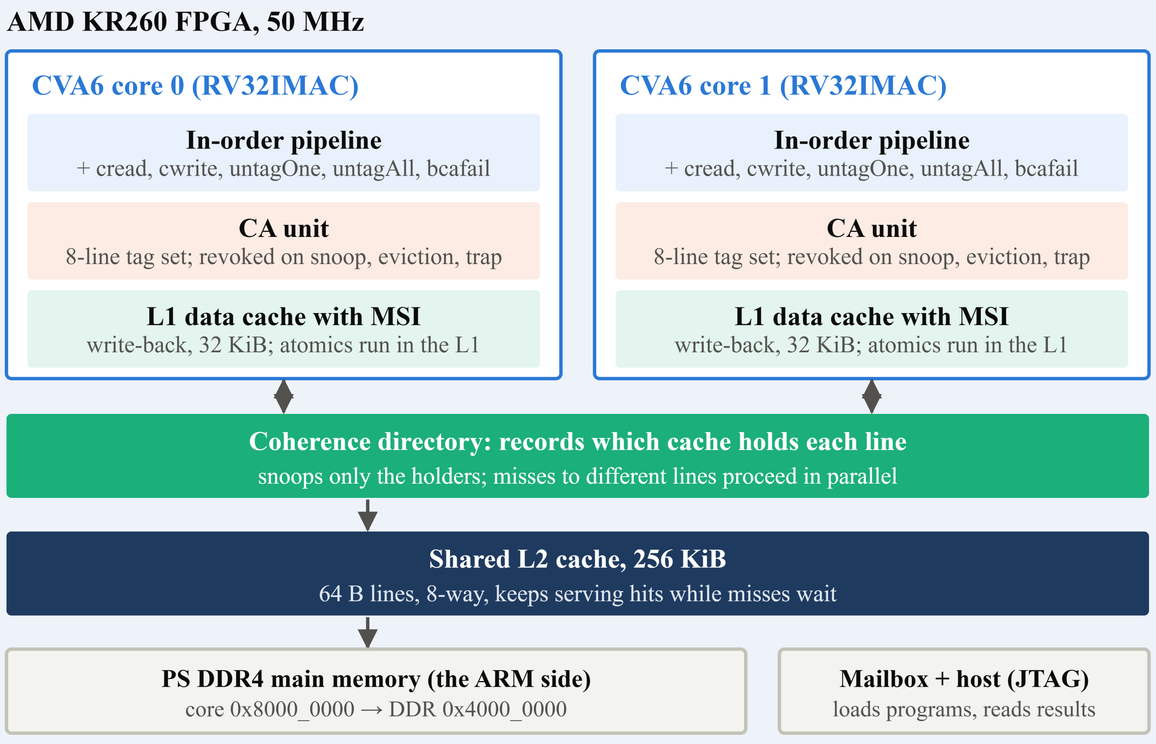


Figure 4. Our two-core system on the AMD KR260. To CVA6 we added the coherent L1 data caches, the MSI directory, the CA units (with `bcafail`) and the shared L2.

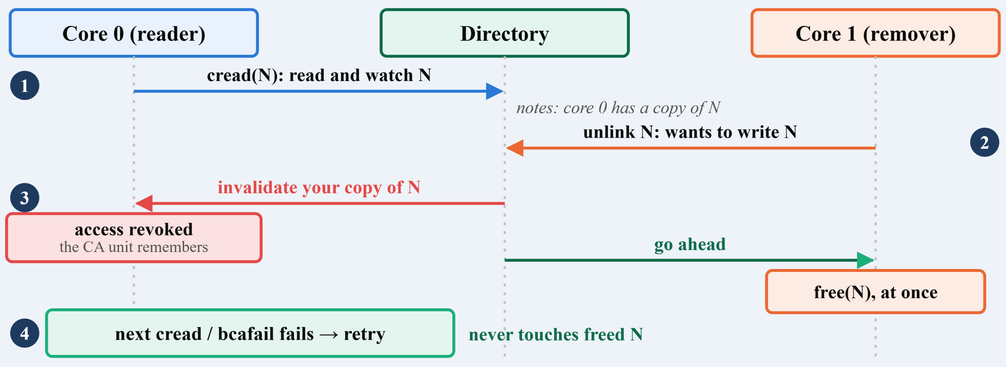


Figure 5. CA reuses the coherence messages the caches already exchange: the remover's write invalidates the reader's copy, and that revokes its access.

5 From paper to hardware: challenges and lessons

Turning a simulated idea into a working chip meant solving problems the paper never faced.

What we had to solve	What we learned
Deadlock A cache always answers the other cache without waiting for anything itself.	**Don't trust stale repositories** OpenPiton (an old CVA6) failed us, so we built our own MSI caches, directory and L2.
Atomic cwrite Check and store happen together, at commit, with the line held exclusively.	**Hardware is finite** 117k LUTs fit 2 cores, so 32-bit RISC-V. Tooling: Vivado 2023.2 licence, JTAG, Windows + WSL.
Speculation CA operations take effect only when the instruction commits.	**Simulators can hide bugs** An L2 handshake bug showed only on the board; after the fix, simulator = board exactly.
No flag register in RISC-V The paper assumes x86 EFLAGS; we added a branch instruction instead: bcafail.	**Faster is not always better** Epochs win read-heavy speed, CA keeps 0 nodes waiting; adding the L2 changed the answer.

6 Evaluation

How we know it works

Cache unit tests	L2 unit tests	Full-system sim	KR260 board
✓ 1,500/1,500	✓ 200/200	✓ 72/73	✓ 1,800
random two-cache simulations pass	random, with slow and stalling DDR	RISC-V ISA tests*, all CA tests pass	benchmark runs, no structure corrupted

26 of 27 deliberately injected bugs are caught. Negative control: with plain loads the board reads freed memory; with `cread` it never does. *The failing ISA test is also disabled upstream.

Setup

The four benchmarks of [1] (lazy list, BST, stack, hash table) with 0% and 100% updates, on two cores with 16 B L1 lines and a shared 256 KiB L2 (the paper: up to 32 threads, 64 B lines, a shared L2), on the **KR260 board** and in a **simulator of the same design**.

Results

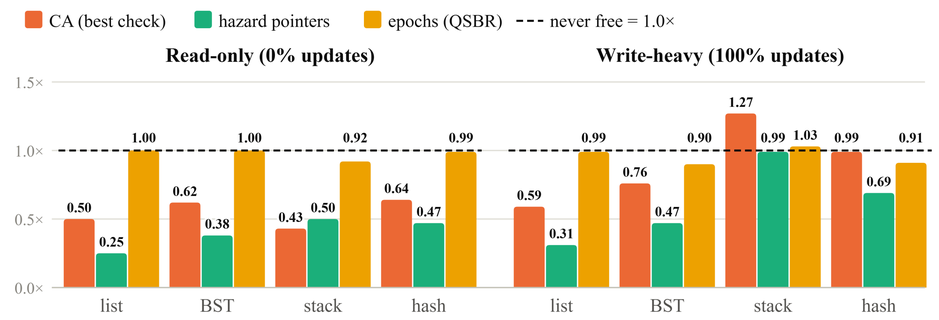


Figure 6. Throughput on the board relative to never freeing memory (higher is better). CA uses its best check method for each workload.

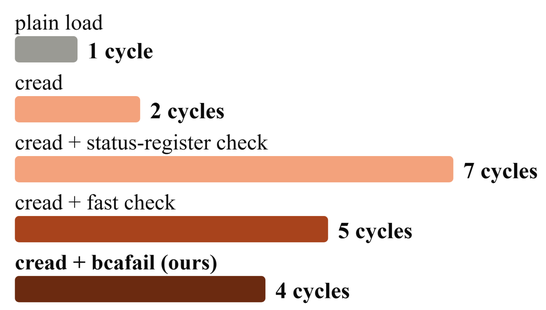


Figure 7. Cycles for one protected read.

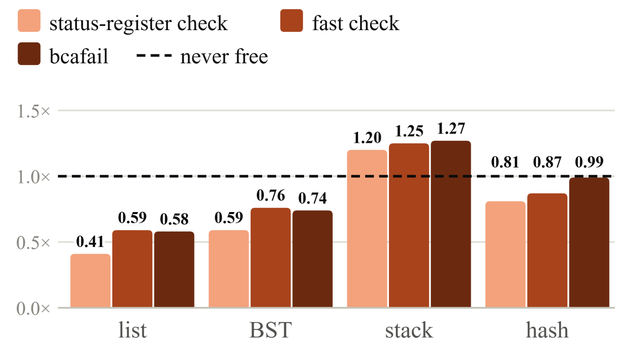


Figure 8. CA throughput by check method (board, 100% updates, relative to never freeing). `bcafail` wins when failures are frequent.

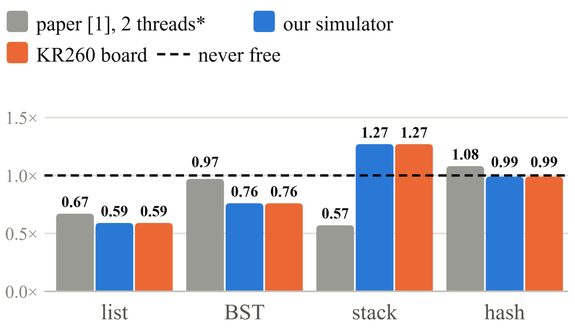
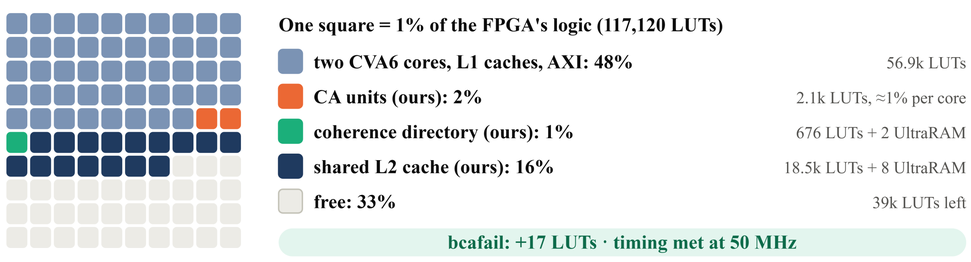


Figure 9. CA (ours: best check) vs. never freeing, 100% updates. Our simulator and board agree exactly (all 144 configurations). *Paper: read off its plot between 1 and 4 threads.

- **Under contention CA is fastest:** 1.27× on the stack, 0.99 on the hash table, ahead of epochs (1.03, 0.91) and hazard pointers (0.99, 0.69).
- **CA frees every node at once:** 0 left waiting, against 20–34 (hazard pointers) and 102–202 (epochs).
- Read-heavy work: epochs are fastest on 2 cores, as in [1] at low thread counts.
- **The shared L2** cut misses from 54 to 33 cycles and made runs up to 2.26× faster, but also exposed CA's per-read cost.

Hardware cost



7 Conclusions

We built a dual-core RISC-V processor with Conditional Access, directory coherence and a shared L2 cache on an AMD KR260, and it **works on real hardware**: 1,800 benchmark runs, no corrupted data structure. CA **frees every node immediately**, and with our `bcafail` branch it is the **fastest scheme under contention** (1.27× never-freeing on the stack), for about 1% of the FPGA per core. A cycle-exact simulator of the same design matches the board exactly. Against the paper at 2 threads we are close on the list and hash table, and differ most on the stack. *Next:* more cores, to test CA where [1] shows its largest gains.

Code and data: `github.com/turing-house-community/multicore-on-fpga`

References. [1] A. Singh, T. Brown, M. Spear. Efficient hardware primitives for immediate memory reclamation in optimistic data structures. arXiv:2302.12958, 2023. [2] F. Zaruba, L. Benini. The cost of application-class processing. *IEEE TVLSI*, 2019. [3] M. M. Michael. Hazard pointers. *IEEE TPDS*, 2004. [4] T. E. Hart et al. Performance of memory reclamation for lockless synchronization. *JPDC*, 2007.

Acknowledgements. We thank Prof. Ajay Singh for his guidance, and the Department of Computer Science and Engineering, IIT Gandhinagar, for the AMD KR260 board and tools.